MSc Project Summative Assignment 3 Reporting - Student ID – 24038155

Week 2

Hi Beren

Well it's finally here and I have to say I'm excited to see how this project turns out

Over the first week i've given a lot of thought to what you said, the last we spoke about going into more detail about the methodology, i will be adopting throughout my project.

For example how I will determine a baseline from which to draw comparison from? And how can I establish a method of comparing results e.g. what criteria can be established with which to make the comparisons.

In my Project Plan I have established a pipeline from natural language to flow diagram and the steps e.g. NL -> Pseudocode -> Algorithm -> AST -> Flow Diagram -> Python code.

The reasoning behind this is to be able to compare at these key stages so I may generate some useful data endpoints with which to make an analysis.

Firstly to be able to find a baseline my approach will be to generate a truth score from the NL to Pseudocode, for example let's say I have the following sentence:

Write me a program to display "Hello World".

From this sentence I can produce a score based on a series of questions based on the following criteria for the produced pseudocode:

- Semantic Correctness
- Pseudocode Correctness
- Syntax Quality
- Readability
- Algorithm Completeness

With a score of 1 – 10 a total score of 50 points per produced pseudocode can be obtained and doubled for a final percentage mark for comparative analysis.

The Following AI models have been selected

Model	**Type**
ChatGPT 5.5	Proprietary
Claude sonnet 4.6	Proprietary
Gemini 3 flash	Proprietary
DeepSeek	Open
Groq llama-3.3-70b-versatile	Open

Model	**Type**
Qwen 3.6 plus	Open

The reasoning behind my selection is for full academic transparency a mix of propriety and open source models should be adopted and compared allowing for a broad overview of characteristics and associated functionality.

To begin with into each AI model I typed the identical request:

Create and display 60 natural language algorithmic tasks and increment complexity every 10

And recorded the outputs, my intention is to choose 2 tasks from each complexity tier listed from each AI model to give an even spread of tasks so that no bias will be present in the task selection process and allowing for an even distribution for research purposes. Then a final list of tasks will be compiled avoiding duplication.

In the coming weeks I will create the baseline pseudocode for comparison so that I can evaluate against the questions for a truth score comparison

As the weeks pass I will update the methodology to include the relevant stages as outlined in the project plan document

At present I am on schedule.

Any feedback would be appreciated

Kind regards

Alan Johnson

p.s. I will email this to you

Week 4

Hi Beren

Below is a breakdown of weeks 3 and 4:

To begin with after I had compiled the list of 60 algorithms from each of the 6 AI models and was reading through them I noticed a disconnect between the various difficulty levels from some of the AI outputs this troubled me as I feel to be impartial in my analysis alignment between the models must be maintained otherwise the study would not be fair across the board so I changed the parameters first raising the difficulty every 12 items and stating the difficulty must start at beginner and be limited to MSc level 7.

Below is the worded request:

- Create and display 60 natural language algorithmic tasks and increment complexity every 15 limiting difficulty from beginner to MSc level 7

This helped a little so I then chose to raise the difficulty every 15 giving 4 tiers; all but 2 were then aligned so ive chosen to select 4 from each tier and then 2 randomly from the top tier giving me 50 algorithms with which to undertake this study. The list will be included at the end of this report in the *Appendix A* section. Getting to this stage took just under a week as I had decided to choose algorithms that were not being repeated in the other lists where possible.

I then started to begin the process of creating a pseudocode vocabulary to be able to make a baseline comparison with; after some thought I chose this vocabulary to be entirely comprised of tokens e.g. [Print} etc.. my thought process with this decision is that however you write pseudocode the simplest form will be a lemmatised version and tokenisation allows me to do just that and stringing essential ones together can produce some complex algorithms; by no means is this an exhaustive list and I'm still adding to is as I find another situation that demands it, I have included the list in *Appendix B.*

I have also began to write the python code for the algorithms in the final list and the pseudocode that will accompany it for the analysis below is the first one.

1. Determine if 2 strings are anagrams of each other

```
string = "1a23"

string1 = "".join(char for char in string if char.isalpha()).lower()

string = "45a6"

string2 = "".join(char for char in string if char.isalpha()).lower()

list1 = list(string1)

list2 = list(string2)

if (len(list1) == 0) or (len(list2) == 0) or (len(list1) != len(list2)):

    print("string1 and string 2 are Not anagrams of each other")

    quit()

list1.sort()

list2.sort()

while len(list1) > 0:

  x = list1.pop()

  y = list2.pop()
```

```
    if (x != y):
        print("string 1 and string 2 are Not anagrams of each other")
        quit()
print("string 1 and string 2 Are anagrams of each other")
```

Here is the corresponding tokenised pseudocode

[Start][Program]

[Var][Name]->string[Equals][Input][String][Source][Data]

[Var][Name]->string1[Equals]

[All][Letter][Characters][In][Var][Name]->string [Converting][Data][To][Lowercase]

[Var][Name]->string[Equals][Input][String][Source][Data]

[Var][Name]->string2[Equals]

[All][Letter][Characters][In][Var][Name]->string [Converting][Data][To][Lowercase]

[If][Length]->string1[Or]->string2[Equals][Zero][Or][If][Length]->string1[Not][Equals][Length]->string2[Print][String]->string1 and string 2 are Not anagrams of each other[Quit]

[Cast][Var]->string1[As][List][Name]->list1

[Cast][Var]->string2[As][List][Name]->list2

[Sort][List]->list1[Asc]

[Start][Loop]

[While][If][Length][List]->list1[Greater][Than][Zero]

[Var][Name]->x[Equals][Pop][Data][From][List]->list1

[Var][Name]->y[Equals][Pop][Data][From][List]->list2

[If]->x[Not][Equal]->y[Print][String]->string 1 and string 2 are Not anagrams of each other[Quit]

[End][Loop]

[Print][String]->string 1 and string 2 Are anagrams of each other

[End][Program]

Here is a worded version of the above pseudocode

source string = input string

remove all characters except letters from source string convert to lowercase and assign to variable string1

source string = input string

remove all characters except letters from source string convert to lowercase and assign to variable string2

if length of list1 equal to zero or length of list 2 equal to zero or length of list1 not equal to length of list2 print "string 1 and string2 are Not anagrams of each other" and quit otherwise

split string1 characters into a list named list1

split string2 characters into a list named list2

sort list1

sort list2

While length of list1 > 0

x equals pop list1

y equals pop list2

if x not equal to y return return "string 1 and string 2 are Not anagrams of each other" and quit

End While

print "string 1 and string 2 Are anagrams of each other"

My reasoning with the tokenised approach is that as you can see from the above example semantic alignment is easily noticed as each step of the program has been detailed in a lemmatised fashion and can also be transcribed into a AST very easily encapsulating the program structure as well as semantic hierarchy, I think this approach will give the most

accurate results when used as a baseline for comparison evaluation of final outputs from the AI models.

This process will be the most time consuming of the entire project.

At present I am on schedule.

Any feedback would be appreciated

Kind regards

Alan Johnson

p.s. I will email this to you

Appendix A

Tier 1
Gwen

1. Determine whether two given strings are anagrams of each other.
2. Convert a non-negative decimal integer into its binary string representation.
3. Determine whether a given year satisfies the leap year rules.
4. Find the intersection of two sorted arrays and return the common elements.

Groq

1. Remove punctuation from a sentence.
2. Find the length of a given string.
3. Split a sentence into individual words and store in a list.
4. Sort a list of words in alphabetical order.

Gemini

1. Given an unsorted list of integers and a target value T, find the exact position of T in the list, or report that it doesn't exist.
2. Scan a paragraph of text and count how many times each individual vowel (a, e, i, o, u) appears.
3. Given a list of IP addresses that accessed a server today, remove all duplicates and return only the unique IP addresses.

4. Given a square grid of numbers ($N \times N$), calculate the sum of the numbers that lie on the main diagonal from top-left to bottom-right.

DeepSeek

1. Reverse a string without using built-in reverse functions.
2. Find the sum of all even numbers in a list.
3. Find the second largest in an array.
4. Find GCD of two numbers using Euclidean algorithm.

Claude

1. Compute the sum and average of a list of numbers.
2. Count the frequency of each character in a string.
3. Swap two variables without using a temporary third variable.
4. Determine all divisors of a given positive integer.

ChatGPT

1. Create a process that determines whether a number is even or odd.
2. Design a procedure that computes the factorial of a positive integer.
3. Write a process that converts numerical scores into grade categories such as Pass, Merit, or Distinction.
4. Create a process that checks whether a password contains at least 8 characters, one number, and one uppercase letter.

Tier 2

Gwen

1. Find the length of the longest substring without repeating characters in a given string.
2. Perform level-order traversal on a binary tree and return node values grouped by depth.
3. Rotate a square 2D matrix 90 degrees clockwise in-place.
4. For a fixed window size sliding across an array, return the maximum value within each window position.

Groq

1. Build a text classification model to categorize text into predefined categories.
2. Create a topic modelling system to extract underlying topics from a large corpus.
3. Develop a co-reference resolution system to identify relationships between pronouns and nouns.
4. Build a question answering system to answer user queries based on a given text.

Gemini

1. Implement a method to find the exact square root of a non-negative integer using binary search, rounded down to the nearest integer, without using built-in math functions.
2. Find all elements that appear in two separate unsorted lists, ensuring the result contains no duplicates.
3. Design a memory buffer that holds a fixed number of items. When it runs out of space, it must automatically discard the item that hasn't been accessed for the longest time.
4. An ordered list of numbers was rotated at an unknown pivot point (e.g., [4,5,6,1,2,3]). Find the index of a target element in this list in logarithmic time.

DeepSeek

1. Implement a stack using an array (push/pop/peek).
2. Find the first non-repeating character in a string.
3. Detect cycle in a singly linked list (Floyd’s cycle detection).
4. Implement a queue using two stacks.

Claude

1. Implement bubble sort and count the number of swaps performed
2. Find all pairs in an array that sum to a given target value
3. Solve the Tower of Hanoi problem for N discs
4. Find the contiguous subarray with the maximum sum (Kadane's algorithm)

ChatGPT

1. Create instructions for adding two matrices of equal size.
2. Develop a process for multiplying two matrices.
3. Create an algorithm that updates inventory quantities after purchases and deliveries.
4. Develop a recursive process that searches through nested folders to locate a specific file type.

Tier 3

Randomly Selected

1. Create a pathfinding solution that guides a user through a maze using graph traversal.
2. Develop algorithms enabling autonomous drones to coordinate movement while avoiding collisions.

[Total 50]

Appendix B

[WHITESPACE & FORMAT]

[Carriage]

[Tab]

[Space]

[Comma]

[Colon]

[Semicolon]

[Quote]

[Singlequote]

[Parentheses]

[Comment]

[Block]

[Length]

[TEXT & LANGUAGE]

[Character]

[Charset]

[String]

[Text]

[Word]

[Sentence]

[Paragraph]

[Section]

[Letter]

[Noun]

[Verb]

[Adjective]

[Pronoun]

[Adverb]

[Personal]

[Possessive]

[Vowel]

[Consonant]

[Pattern]

[Match]

[Replace]

[Split]

[Delimiter]

[Remove]

[DATA TYPES]

[Var]

[Variable]

[Constant]

[Int]

[Float]

[Double]

[Boolean]

[Primitive]

[Object]

[Class]

[Function]

[Method]

[Property]

[Parameter]

[Parameters]

[Type]

[Cast]

[As]

[All]

[With]

[Null]

[Undefined]

[Empty]

[BOOLEAN & LOGIC]

[True]

[False]

[And]

[Or]

[Xand]

[Xor]

[Not]

[Equals]

[Greater]

[Less]

[Than]

[Compare]

[Is]

[CONTROL FLOW]

[If]

[Then]

[Else]

[While]

[Do]

[Until]

[For]

[Each]

[Step]

[Loop]

[Iterate]

[Nest]

[Recurs]

[Return]

[Try]

[Catch]

[Except]

[Quit]

[ARITHMETIC]

[Add]

[Sub]

[Mult]

[Div]

[Inc]

[Dec]

[Modulus]

[Quotient]

[Sum]

[Average]

[Product]

[Fraction]

[Numerator]

[Denominator]

[Ratio]

[Percent]

[Abs]

[Root]

[Integral]

[Differential]

[Calculate]

[Result]

[NUMBER SYSTEMS]

[Num]

[Decimal]

[Binary]

[Hexadecimal]

[Octal]

[Base]

[e]

[x]

[Positive]

[Minus]

[Zero]

[Rand]

[BITWISE & LOW LEVEL]

[Bit]

[Shift]

[Control]

[Keycode]

[Upper]

[Lower]

[Limit]

[State]

[On]

[Off]

[DATA STRUCTURES]

[List]

[Array]

[Queue]

[Deque]

[Stack]

[Node]

[Edge]

[Graph]

[Map]

[Table]

[Row]

[Column]

[Field]

[Record]

[Index]

[Pointer]

[Reference]

[Linked]

[Ordered]

[Unordered]

[Sequence]

[Series]

[Tree]

[DATA OPERATIONS]

[Push]

[Pop]

[Shift]

[Unshift]

[Insert]

[Delete]

[Remove]

[Update]

[Create]

[Rename]

[Move]

[Copy]

[Cut]

[Paste]

[Save]

[Edit]

[Find]

[Search]

[Track]

[Seek]

[Put]

[Append]

[Overwrite]

[Sort]

[Asc]

[Desc]

[SET THEORY]

[Set]

[Subset]

[Union]

[Intersection]

[Member]

[Element]

[FUNCTIONAL RELATIONS]

[From]

[To]

[In]

[Out]

[Of]

[By]

[With]

[Without]

[Using]

[According]

[PROGRAM STRUCTURE]

[Define]

[Code]

[Source]

[Dest]

[Input]

[Output]

[Start]

[Stop]

[Begin]

[End]

[Current]

[Next]

[Before]

[After]

[Active]

[Scope]

[Static]

[Protected]

[Private]

[This]

[Inherits]

[Extends]

[Bind]

[FILE SYSTEM & IO]

[File]

[Folder]

[Name]

[Open]

[Close]

[Read]

[Write]

[Url]

[Mimetype]

[Format]

[DATABASE & NETWORK]

[Database]

[Api]

[Connect]

[Link]

[Script]

[Meta]

[Style]

[Doctype]

[Lang]

[TIME]

[Zone]

[Time]

[Hour]

[Minute]

[Second]

[Day]

[Month]

[Year]

[POSITION & DIRECTION]

[Top]

[Bottom]

[Left]

[Right]

[Center]

[Above]

[Below]

[Forward]

[Back]

[Next]

[Reverse]

[Direction]

[Directed]

[Adjacent]

[Opposite]

[GEOMETRY & GRAPHICS]

[Shape]

[Polygon]

[Circle]

[Square]

[Rect]

[Rhombus]

[Ellipse]

[Point]

[Pixel]

[Line]

[Coordinate]

[X]

[Y]

[Z]

[Axis]

[Rotate]

[Reflect]

[Translate]

[Clockwise]

[Anti]

[Straight]

[Curved]

[Orthogonal]

[Perpendicular]

[COLOUR & DISPLAY]

[Foreground]

[Background]

[Font]

[Colour]

[Red]

[Green]

[Blue]

[White]

[Black]

[Alpha]

[Visible]

[Hidden]

[MATH FUNCTIONS]

[Log]

[Sin]

[Cos]

[Tan]

[Cot]

[Sec]

[Cosec]

[Inv]

[Exp]

[Ln]

[Arc]

[Deg]

[Rad]

[Grad]

[Mean]

[Median]

[Probability]

[Proportional]

[GRAPH THEORY & VISUALISATION]

[Chart]

[Pie]

[Bar]

[Legend]

[Title]

[Label]

[Depth]

[Breadth]

[DOCUMENT STRUCTURE]

[Page]

[Heading]

[Header]

[Footer]

[Artifact]

[Component]

[SPATIAL & MEASUREMENT]

[Long]

[Unit]

[Measure]

[Offset]

[Position]

[Align]

[Justify]

[Prefix]

[Suffix]

[Origin]

[MISCELLANEOUS]

[Yes]

[No]

[All]

[Any]

[Once]

[Duplicate]

[Cycle]

[Range]

[Mode]

[Max]

[Min]

[Top]

[Bottom]

[Longitudinal]

[Transverse]

[Fractal]

[Data]

[Image]

[Id]

[Call]

[Page]

Week 6

Hi Beren

Below is a breakdown of weeks 5 and 6:

After our last communication I read a few academic papers on the limitations and concerns of tokenisation, which has caused me to reach the conclusion that fully tokenising each algorithm is adding a layer of complexity that really doesn't need to be there and as such I am now limiting tokenisation to pre-built in functionality an example of which is the [Print] statement, my reasoning for this is detailed that statistical inaccuracies are introduced by way of "a tokenizers consistency is implicitly is implicitly or explicitly assumed" (Gastaldi et. al., 2025), the paper also states there are many ways that conditions in practice do fail.

I must say I found this paper very informative on how the tokenisation process is mapped and I found the definitions very insightful.

I believe I had the right idea with full tokenisation but over thought it slightly.

I intend to read a few papers on suggested pseudocode grammar in the next few weeks.

And using the set of tokens I have previously generated I intend to shrink it to only include the usual python built in functionality and define a more natural language centric adoption for the pseudocode grammar, which will allow for a better alignment as a baseline comparison.

I have also started the process of writing the python code for the 50 algorithms chosen

Oh I changed one of the algorithms in the final list number, now algorithm 41 is:
Solve the N Queens Problem for an arbitrary board size N
This is because the bubble sort algorithm is too similar to an earlier algorithm

At present I have completed the first 48 algorithms in doing so I have tried to program each one with a clearly defined set of conditions namely:

- Use the simplest method to achieve an end result
- Try to avoid pythons built in functionality
- Avoid recursion and python shortcuts unless the algorithm specifically requires them
- Write each algorithm in a consistent style
- Try to represent each algorithm as clearly as possible

This will ensure consistency in any analysis and rule out academic bias by way of not favouring a particular approach over others when making comparisons.

I have also started to think about the poster and the information I will be including.

The next 2 weeks will consist of the following activities:

- Complete the remaining programming tasks
- Read academic research on pseudocode grammar
- Generate a more appropriate pseudocode grammar
- Continue to plan the poster

Kind Regards
Alan Johnson

References:

Gastaldi, J.L., Terilla, J., Malagutti, L., DuSell, B., Vieira, T. and Cotterell, R., 2025. The foundations of tokenization: statistical and computational concerns. In: International Conference on Learning Representations (ICLR).

Week 8

Hi Beren

Firstly apologies for this report reaching you a day late, I had to work.

Below is a breakdown of weeks 7 and 8:

During week 7 and 8 I completed the remaining python code tasks and began considering a pseudocode grammar.

While researching the possible alternatives it soon became clear that there is no standard pseudocode grammar and rather than trying to reinvent the wheel I resolved to refer to the Cambridge teacher's guide (Cambridge University Press & Assessment, 2023), as a baseline for its generation.

Here is the first program and its pseudocode representation:

1. Determine if 2 strings are anagrams of each other

[python code]

```
string = "1a23"
string1 = "".join(char for char in string if char.isalpha()).lower()
string = "45a6"
string2 = "".join(char for char in string if char.isalpha()).lower()
list1 = list(string1)
list2 = list(string2)
if (len(list1) == 0) or (len(list2) == 0) or (len(list1) != len(list2)):
      print("string1 and string 2 are Not anagrams of each other")
      quit()
list1.sort()
list2.sort()

while len(list1) > 0:
   x = list1.pop()
   y = list2.pop()
   if (x != y):
      print("string 1 and string 2 are Not anagrams of each other")
      quit()
print("string 1 and string 2 Are anagrams of each other")
```

[pseudocode]

```
string <- "1a23"
string1 <- ""
FOR EACH char IN string DO
    IF char IS A LETTER THEN
        string1 <- string1 + LOWERCASE(char)
    ENDIF
ENDFOR

string <- "45a6"
string2 <- ""
FOR EACH char IN string DO
    IF char IS A LETTER THEN
        string2 <- string2 + LOWERCASE(char)
    ENDIF
ENDFOR

list1 <- LIST OF CHARACTERS IN string1
list2 <- LIST OF CHARACTERS IN string2

IF (LENGTH(list1) = 0) OR (LENGTH(list2) = 0) OR (LENGTH(list1) !=
LENGTH(list2)) THEN
    OUTPUT "string1 and string2 are Not anagrams of each other"
    STOP
ENDIF

list1 <- SORT(list1)
list2 <- SORT(list2)

WHILE LENGTH(list1) > 0 DO
    x <- REMOVE LAST ITEM FROM list1
    y <- REMOVE LAST ITEM FROM list2

    IF x != y THEN
        OUTPUT "string1 and string2 are Not anagrams of each other"
        STOP
    ENDIF
ENDWHILE
```

Any thoughts would be appreciated.

I have also started to create the poster; the approach I am considering is a guided explanation of the process of NLP explaining the purpose of tokenisation and lemmatisation in the parsing of NL and the generation of algorithmic representations such as pseudocode and AST's . I will then introduce the idea of executable flow diagrams and discuss the usages and possible advantages for example a flow diagram is a representation of a computer program and armed with that information a computer program can be compiled in any computer language ensuing semantic alignment without the need to extra processes to maintain alignment. Key aspects of a given language can then be measured and a decision for choosing a particular language can then be made based on those strengths and weaknesses.

Another interesting perspective is for educational purposes, prospective computer scientists can learn by focusing on creating a flow diagram and compiling it into various programming languages; then by seeing the different ways these languages accomplish the same tasks, can find various methods to accomplish a given task this can then encourage the generation of an observable approach to creating algorithms.

Again any thoughts would be appreciated.

Kind Regards

Alan Johnson

References

Cambridge University Press & Assessment (2023) *Pseudocode Guide for Teachers: Cambridge International AS & A Level Computer Science 9618 (For exams in 2026)*. Cambridge: Cambridge University Press & Assessment. Available at: https://www.cambridgeinternational.org/Images/697401-2026-pseudocode-guide-for-teachers.pdf

Week 10

Hi Beren

Just a short report this time:

In the past 2 weeks I have solely been focused on the required poster, as per our last communique I started with a discussion of the process and the steps that are being undertaken but it just didn't work for me it lacked any real flare so I thought id come at it from another direction and I decided to create it as I would like a web page ive created it in PowerPoint as a single slide I haven't used any voice over as the poster really pops and contains as I originally wanted a description of the pipeline and process I am using except it is now being illustrated as a series of diagrams. The layout uses a header and a 3 column layout with a footer section containing a short bio describing how I came up with the idea of using flow diagrams to maintain a common baseline for various programming languages.

The next 2 weeks are going to encompass finalising the poster design and then onto the process of cataloguing the pseudocode generated by the AI agents.

Kind Regards

Alan Johnson

Week 12

Hi Beren

In the past 2 weeks I have finished and submitted the summative assignment for the project poster, im really pleased with the result if you haven't seen it I have a GitHub repository with the poster and other files. In the remaining week I have completed the pseudocode for the algorithmic tasks and detailed the custom rules used to generate it alongside the Cambridge Teaching Guide.

The reasoning for the changes are 2 fold firstly to aid in readability and secondly to more easily facilitate any compilation process required. This concludes the baseline measurement required for comparative analysis section of this study. The file has been uploaded to GitHub.

During the next 2 weeks I am going to be:

- Asking each AI agent to generate pseudocode for each of the algorithmic tasks and recording the results
- After which I will then ask each AI agent to generate python code for each of the algorithmic tasks and recording the results
- I will then score each of the results based on a set of questions concerned with accuracy, semantic alignment, readability etc.

After this laborious process I will firstly be relieved lol and then can start engineering the parser to be able to generate AST and Flow diagrams

The GitHub repository can be accessed at:

https://github.com/ajohnson-2210/Final_Project

Kind Regards

Alan Johnson

Week 14

Hi Beren

First can I have clarification on the final submission date for the project originally I put the date of 31 oct in my calendar but on canvas its now 12 oct thx in advance

In the past 2 weeks I have completed the pseudocode grammar it is in the github repo called:

Cambridge Style Pseudocode Grammar - AJ_24038155 Dialect v1.0

I have also nearly finished the software engineering portion of the project I made the decision to hold off on the questions and scoring required for the analysis portion of the project the pipeline is basically finished but still needs testing and the bugs addressed included in the appendix of this report ive attached the artifacts produced by the pipeline.

At present the pipeline runs and asks what is being input either pseudocode or a drawio flow chart ive included the functionality to add a flowchart to illustrate the usefulness of parsing a flow diagram and have a generated AST and a compiled Python program. Once the pseudocode option is chosen it askes which parser option to choose there are 3 a strict mode a general mode these alter the pseudo code to more align with my dialect and the 3rd option does not alter the code and attempts to parse it and generate the artifacts. I have to say im pleased with the results so far, im sure you will be too but as always any suggestions or constructive criticism is welcome.

The following 2 weeks will be streamlining the pipeline so that im not testing against my pseudocode grammar but simply using it as a baseline comparison.

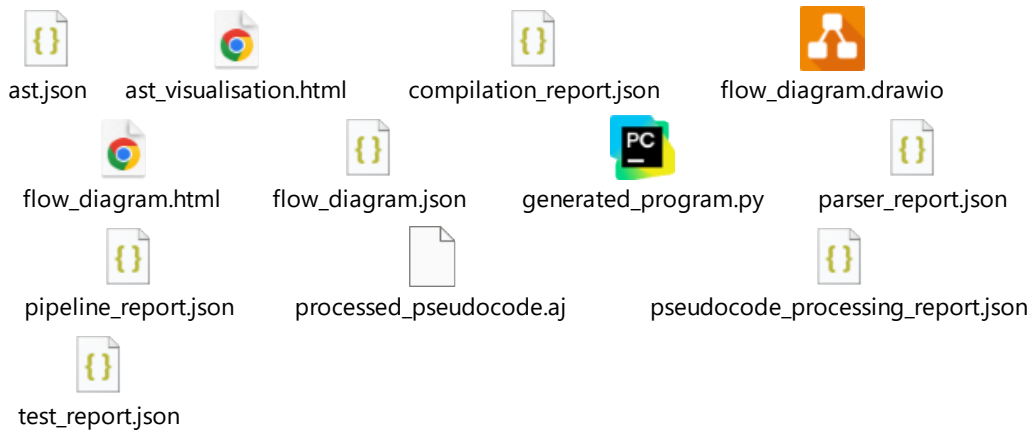
The GitHub repository can be accessed at:

https://github.com/ajohnson-2210/Final_Project

Kind Regards

Alan Johnson

Appendix



Week 16

Hi Beren

I received my mark for the poster; I was a bit disappointed as I lost marks for not having included any references, which was annoying as I originally had a references section and removed it because my short biography went a little over the window size so I removed it and lengthened the footer section. Ah well not to worry a lesson learned.

In the past 2 weeks I have been focused on the normalisation process and filtering through instances where the inputted pseudocode fails to parse and needs to be interpreted into a canonical form that maintains the semantic structure of the original pseudocode.

Then and only then will the audit process and subsequent unit tests be allowed to run and permit parsing into the next stage of that of an AST.

Here is an example snippet of the produced JSON during this audit of the code, it details the rule that is being implemented and the branch of that rule, I originally was detailing each rule separately and they were being identified with an ID but I found there was a lot of repetition so I chose to alter the design slightly and have rule families with branch sub sections.

As you can see then the source of the parsing failure is recorded and the canonical replacement that the parse can recognise for the compiler to be able to parse; detailed also is the evidence and therefore justification of this amendment, also in the confidence the certainty that this being a known syntactic misalignment with expected behaviour is measured and recorded, this is based on the rule set that have been necessary to align differing forms of tokenisation that equate to the same expected outcome, and finally a record of whether there has been a change to the semantic structure of the algorithm.


```
{

  "rule_family": "LIST_SELF_ADDITION",

  "branch": "SINGLE_ELEMENT_RHS",

  "source": "files <- files + [item]",

  "canonical": "APPEND item TO files",

  "evidence": [

    "files is established as a list",

    "RHS is a singleton list literal"

  ],
```

"confidence": "structurally certain",

"semantic_change": false

}

The following 2 weeks will be to complete this task of which I am approx. 80% through and then based on the evidence being produced by the internal parsing audit I will create the marking criteria and list of scoring questions to analyse this process, I feel basing the analysis on this aspect of the transformation process will yield the most compelling data.

As of yet I haven't uploaded any of the versions to the GitHub repo as there are quite a few and id like to do that once I am satisfied the pipeline works as expected.

The GitHub repository can be accessed at:

https://github.com/ajohnson-2210/Final_Project

Kind Regards

Alan Johnson

Week 18

Hi Beren, firstly sorry for the report being a day late I had to work.

Over the last 2 week I have been ironing out the compiler bugs which I am pleased to say is complete. These consisted of issues related to the various syntaxes associated with the various types of pseudocode included in the corpus of algorithm responses from the AI agents. This has been quite a revelation to me with respects to how this resource is not being governed; I intend to discuss this fact and the possible benefits in my dissertation.

I have also began implementing the analysis aspect of the project my approach is 2 fold, firstly as the pseudocode is input into the pipeline; during parsing under the strict option the algorithm is tokenised and canonicalised into the established dialect that has been developed and in doing so audited for any various mismatch cases at present these; as discussed in my last report are documented and output as an artefact, this information will be used by the pipeline to score on the criteria established and a further artefact will be produced giving a score that can be assessed.

And secondly a questionnaire that I will fill in related to each algorithm will be produced giving a human aspect to this study and can also be utilised in a larger scenario if that were to be a consideration the questions and scoring methods are grounded academically.

I also wanted to inform you I will be on holiday from the 25th for 2 weeks I know the timing isn't great but this was booked last year so I may miss that report or at least it will be included in the final submission.

The GitHub repository can be accessed at:

https://github.com/ajohnson-2210/Final_Project

Kind Regards

Alan Johnson

Week 20

Hi Beren, firstly sorry for the report being late I had to work on the night shift and got my weeks mixed up.

Over the last 2 week I have been fine tuning the pipeline for the analysis and ive implemented the human aspect to the evaluation and included it into the final analysis that will be included for consideration.

I have also developed a number of dashboards to display the data for comparison and to enable the questionnaire so that I can fill in the human evaluation related to each algorithm.

The scoring is then added into the calculation and averaged to give a fairer scoring metric.

I have also mapped out the relevant point for the video presentation.

Over the next 2 weeks I will be scoring and assessing the individual algorithms pseudocode results from the AI agents and recording the video.

I am on holiday from the 25th Sep to 9th Oct I will post that weeks report when I get back.

The GitHub repository can be accessed at:

https://github.com/ajohnson-2210/Final_Project

Kind Regards

Alan Johnson

Week 22

Hi Beren, Over the last 2 week I have been completing the human evaluation section of the analysis of the produced pseudocode.

I included this aspect to the study for the following reasons:

- To ensure the automated scoring was correct
- To discover any trends that may be attributed to pipeline bugs
- To ensure a fair and un-bias overview of the data

Upon completion of the review a number of system bugs became apparent these are listed below:

- Nested loops are flagged as errors in the flow diagram, due to the test suit interpreting the absence of a return path as an error; upon review of the flow diagram in question, the problem became apparent, these instances were marked as correct in the human review
- As with nested loops also class methods had the same problem, again upon review these were marked as correct
- Some algorithms needed coded examples to test them e.g bfs required a node structured graph implemented and fed into the function for testing purposes this happened a lot during the review with the exception of simpler tasks these were tested and marked accordingly.
- There was also a bug that when the AST and adjacency list was generated and used to create the flow diagram some block nodes were discarded as they were semantic markers only; on occasion the test suit read this as a miss match due to node count difference but upon review the diagrams were correct these were marked as correct

I have addressed the bugs in a version update but due to time constraints I don't have enough time to redo the human review of the 300 algorithms each has 16 questions so the data from the study version with the bugs will have to do, ill include all versions in the git hub before submission.

I have also created dashboards to display the scoring of the various AI agents

Due to me being on holiday I have requested a week extension to give me a little extra time to complete the video and dissertation writing.

The GitHub repository can be accessed at:

https://github.com/ajohnson-2210/Final_Project

Kind Regards

Alan Johnson